CRUD API (Create, Read, Update, Delete)

1. Introduction:

- CRUD stands for **Create, Read, Update, Delete**, representing the four basic operations of persistent storage in software systems.
- In web development, **CRUD APIs** define how applications interact with databases through **HTTP methods**.
- CRUD operations are fundamental in **RESTful API design** — each operation maps to a specific HTTP method.

CRUD Operation	HTTP Method	Description
Create	POST	Add new data to the server
Read	GET	Retrieve existing data
Update	PUT / PATCH	Modify existing data
Delete	DELETE	Remove data from the server

2. Importance of CRUD APIs:

- Provide a **standardized structure** for data management.
- Enable communication between client and server applications.
- Allow consistent and scalable interaction with databases.
- Commonly used in web apps, mobile apps, and backend services.

Example:
A student management system might use a CRUD API to:

- Create a new student record.
- Read student details.
- Update grades.
- Delete a withdrawn student's record.

3. CRUD and RESTful API Relationship:

- REST (Representational State Transfer) is a web architecture that uses standard HTTP methods for CRUD operations.
- Each resource (e.g., user, product, order) is identified by a **unique URL (endpoint)**.
- CRUD actions are performed using these endpoints with the appropriate HTTP methods.

Example base URL:
https://api.example.com/students

4. CREATE Operation (HTTP POST):

- Used to **add new data** to the server or database.
- The client sends a **POST request** with data in the body.

Example Request:

```
POST /api/students HTTP/1.1
Host: api.example.com
Content-Type: application/json

{
"name": "Aayush",
"roll": 25,
"course": "Computer Science"
}
```

Example Response:

```
HTTP/1.1 201 Created
Location: /api/students/25

{
"id": 25,
"name": "Aayush",
"course": "Computer Science"
}
```

Explanation:

- The server processes the request, creates a new record, and returns a **201 Created** status.
- The Location header provides the URI of the newly created resource.

5. READ Operation (HTTP GET):

- Used to **retrieve data** from the server.
- It is a **safe** and **idempotent** operation (does not modify data).

Example Request:

```
GET /api/students/25 HTTP/1.1
Host: api.example.com
```

Example Response:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
"id": 25,
"name": "Aayush",
"course": "Computer Science"
}
```

Explanation:

- The server retrieves and returns the resource with the specified ID.
- If the resource does not exist, the server responds with **404 Not Found**.

6. UPDATE Operation (HTTP PUT / PATCH):

- Used to **modify existing data** on the server.
- **PUT** replaces the entire resource, while **PATCH** updates specific fields.

Example PUT Request:

```
PUT /api/students/25 HTTP/1.1
Host: api.example.com
Content-Type: application/json

{
"name": "Aayush Gupta",
"course": "Data Science"
}
```

Example Response:

```
HTTP/1.1 200 OK
{
"id": 25,
"name": "Aayush Gupta",
"course": "Data Science"
}
```

Example PATCH Request:

```
PATCH /api/students/25 HTTP/1.1
Host: api.example.com
Content-Type: application/json
```

```
{
 "course": "Artificial Intelligence"
 }
```

Explanation:

- PUT replaces the entire resource (even unchanged fields).
- PATCH modifies only the provided attributes.

7. DELETE Operation (HTTP DELETE):

- Used to **remove data** from the server.
- It is **idempotent** — deleting the same resource multiple times has the same effect.

Example Request:

```
DELETE /api/students/25 HTTP/1.1
Host: api.example.com
```

Example Response:

```
HTTP/1.1 204 No Content
```

Explanation:

- The resource is deleted.
- The server responds with **204 No Content** (no body needed).

8. CRUD API Workflow Example

Scenario: Managing student records.

Action	HTTP Method	Endpoint	Example
Create new student	POST	/api/students	Adds new record
Get all students	GET	/api/students	Returns all records
Get specific student	GET	/api/students/25	Returns one record
Update student info	PUT	/api/students/25	Updates full record
Delete student	DELETE	/api/students/25	Removes record

9. Status Codes in CRUD APIs:

Operation	Success Code	Failure Code	Meaning
CREATE (POST)	201 Created	400 Bad Request	Resource created or invalid input
READ (GET)	200 OK	404 Not Found	Resource retrieved or missing
UPDATE (PUT/PATCH)	200 OK / 204 No Content	400 Bad Request / 404 Not Found	Updated successfully or invalid
DELETE (DELETE)	204 No Content	404 Not Found	Deleted successfully or resource missing

10. CRUD Example Using cURL:

Create:

```
curl -X POST https://api.example.com/students -H "Content-Type: application/json" -d '{"name":"Aayush","roll":25,"course":"CSE"}'
```

Read:

```
curl -X GET https://api.example.com/students/25
```

Update:

```
curl -X PUT https://api.example.com/students/25 -H "Content-Type: application/json" -d '{"course":"AI"}'
```

Delete:

```
curl -X DELETE https://api.example.com/students/25
```

11. Advantages of CRUD APIs:

- Provide a **structured and predictable** interaction model.
- Easy to integrate with front-end and mobile applications.
- Follow **REST principles**, promoting scalability and modularity.
- Enhance maintainability through clear method-resource mapping.

12. Security Considerations:

- Always use **HTTPS** to protect transmitted data.
- Implement **authentication** (e.g., JWT, OAuth2) to control access.
- Use **input validation** to prevent SQL injection and XSS attacks.
- Log and monitor API usage for anomaly detection.

Summary:

- CRUD represents the four basic data operations: **Create, Read, Update, Delete**.
- Each CRUD operation maps directly to an **HTTP method** in REST APIs.
- CRUD APIs simplify database interaction, improve code organization, and enhance scalability.
- Proper use of **status codes, validation, and security** ensures a reliable and secure API system.